Lab session #8
Deep Learning

Dr Zied M'NASRI

Introduction

In this Lab session, we learn how to implement and train Deep Learning algorithms, such as Deep Neural Networks (DNN) and Convolutional Neural Networks (CNN) to make prediction and also feature extraction. You will need to use:

1. Python tutorial (see Lab session 1 material)
2. Colab tutorial (see Lab session 2 material)
3. Lecture 8 notes
4. Lecture 8 code examples (DNN and CNN examples)
5. Google's Colab or Jupyter Notebooks
6. The files "pima-indians-diabetes.csv" and "elephant.jpg" attached in Lab8 materials

Read and run Tutorials I and II, then use the implemented functions to develop Exercises 1, 2 and 3.

Part I- Deep Neural Networks (DNN)

Tutorial 1- Binary Classification using DNN

In this example, the file '*pima-indians-diabetes.csv'*, to understand how DNN are trained to carry out binary classification. Using the following steps, we build and train a DNN to classify Diabetes data by DNN ((see the file U*B_AI_Lecture8_DNN_Example.ipynb*)

1. Import the libraries and the dataset file

```
# Import libraries
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# load the dataset
from google.colab import files
uploaded = files.upload()
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')

# Visualise the dataset
dataset
```

2. Split the dataset columns into input features (Columns 0 to 7) and labels (Column 8)

```
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
```

3. Build a DNN network architecture containing 3 Dense (fully-connected) layers, containing, respectively, {12,8,1} nodes with the following activation function {`Relu, Relu,Sigmoid`}

```
# Build the DNN network
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

4. Compile the built model and dd the command `model.summary()` to visualise its architecture architecture:

```
# compile the keras model
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12)	108
dense_1 (Dense)	(None, 8)	104
dense_2 (Dense)	(None, 1)	9

Total params: 221 (884.00 B)
Trainable params: 221 (884.00 B)
Non-trainable params: 0 (0.00 B)

Figure 1.1.1. (feedforward) DNN network

5. Split the sleected input and output data into training and testing subsets {`X_train, y_train`} and {`X_test, y_test`} using 80% for training and 20% for testing. Setting the random state to the same value, e.g. 42 allows obtaining the same random split of the dataset each time, leading to the same results each time you train the model.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (614, 8)
X_test shape: (154, 8)
y_train shape: (614,)
y_test shape: (154,)
```

Figure 1.1.2. Sizes of training and testing sets

6. Train the DNN network using the method `model.fit()`. Note that we can adjust the number of epochs (`epochs=150`) and the size of the minibatch (`batch_size=10`):

```
# fit the keras model on the dataset
history = model.fit(X_train, y_train, epochs=150, batch_size=10)
```

```
Epoch 1/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.3675 - loss: 11.4144
Epoch 2/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.4521 - loss: 2.2892
Epoch 3/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5050 - loss: 1.3946
Epoch 4/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5648 - loss: 1.1975
Epoch 5/150
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5110 - loss: 1.1677
```

Figure 1.1.3. Training the DNN model

7. After training the DNN, add the following code to visualise the accuracy and loss curves during training

```
# Plot the model's loss and accuracy
from matplotlib import pyplot as plt
plt.plot(history.history['accuracy'])
plt.title('Accuracy on the training set')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.show()

plt.plot(history.history['loss'])
plt.title('Loss on the training set')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.show()
```

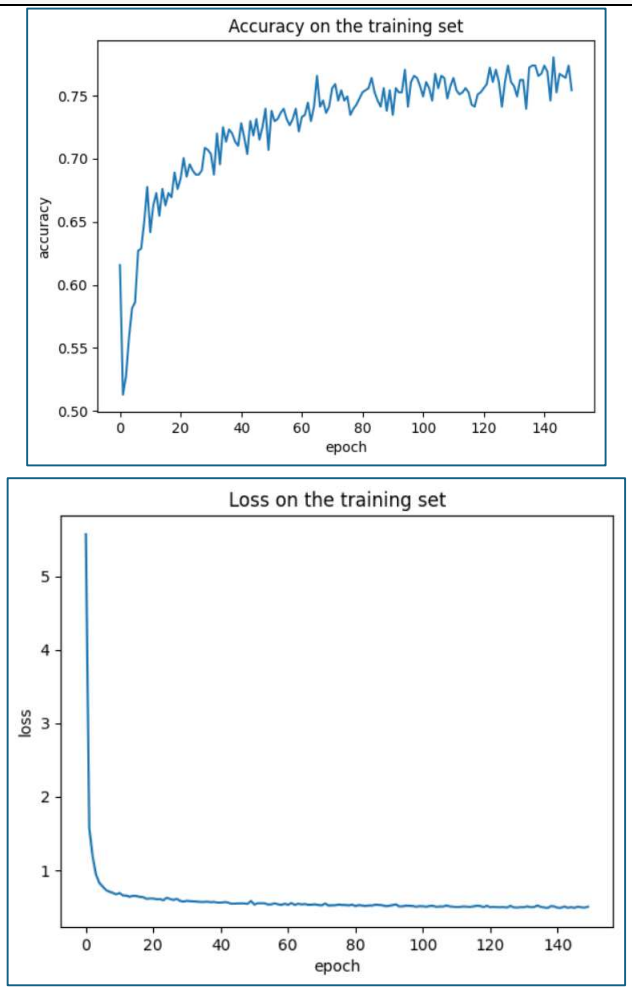


Figure 1.1.4. Accuracy and loss curves

8. Perform binary classification prediction on the testing set using a `threshold = 0.5`

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)
Threshold = 0.5

for i in range(len(y_pred)):
  if y_pred[i] > Threshold:
    y_pred[i] = 1
  else:
   y_pred[i] = 0
```

9. Calculate and print the classification report's metrics (Accuracy, Precision, Recall and F1) for the test set

```
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy: %.2f' % (accuracy*100))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 70.78
              precision    recall  f1-score   support

         0.0       0.70      0.94      0.81        99
         1.0       0.73      0.29      0.42        55

    accuracy                           0.71       154
   macro avg       0.72      0.62      0.61       154
weighted avg       0.71      0.71      0.67       154
```

Figure 1.1.5. Classification report

Exercise #1: Multi-class classification using (feedforward) DNN

In this exercise we aim at designing and training a (feedforward) deep neural network (DNN) model for Iris dataset, i.e. multi-class classification with 3 labels, using the same NN initialisation, training and testing predefined methods used in **Tutorial I**.

Steps

1. **Setup**

   Import the necessary libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
```

2. **Data preparation**

   2.1.Load the Iris dataset, print its head and display its unique labels

```
Iris ddatasets unique targets:  [0 1 2]
```

Figure 1.2.1. Unique target labels of Iris dataset

Hints!

a. To load the Iris dataset use the method `load_Iris()`, and set the argument `as_frame = True`

b. Convert the dataset to a dataframe, using the method `dataframeName = datasetName.df`

c. Display the header (5 first rows) using the method `dataframeName.head()`

d. Get the unique target labels using the method `dataframeName['target'].unique()`

2.2.Split the Split the dataframe into training and features X_train, X_test, and training and test targets y_train and y_test, using a test/train ratio = 0.3 and random state = 42, and print the shape of each split.

```
X_train:  (105, 4)
X_test:  (45, 4)
y_train:  (105,)
y_train:  (45,)
```

Figure 1.2.2. Sizes of the training and testing sets

Hints!

a. Import the function `train_test_split()` from the library `sklearn.model_selection`

b. Apply the function `train_test_split()` to `dataframeName.iloc[:,:-1]` for the features, and `dataframeName.iloc[:,-1]` for the targets

3. Feature and target preprocessing

3.1.Label encoding

Encode the labels using the One-hot encoding scheme, i.e. label 0 →[1,0,0], label 1→ [0,1,0] and label2→[0,0,1]

Hints!

a. Import the method `OneHotEncoder` from the library `sklearn.preprocessing`

b. Create a OneHotEncoder object by using the method `OnehotEncoder()` and setting its argument `sparse_encoder = False`

c. Reshape the training and test targets as follows:
`np.array(y_train.reshape(-1,1))`
`np.array(y_test.reshape(-1,1))`

d. Apply the method `encoderName.transform()` to the reshaped array

e. Print the shape of the encoded targets

```
Encoded training targets shape:  (105, 3)
Encoded training targets shape:  (45, 3)
Encoded_training targets:
 [[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
```

Figure 1.2.3. One-hot-encoded labels

3.2.Feature normalisation

a. Import the method `Normalizer` from the library `sklearn.preprocessing`

b. Create an object `normalizerName` using the method `Normalizer()`

c. Fit the normaliser on X_train using the method `Normaliser.fit()`

d. Normalise X_train and X_test by applying the method `NormaliserName.transform()`

e. Print the normalised features

```
X_train normalised:
 [[0.77242925 0.33706004 0.51963422 0.14044168]
 [0.72366005 0.32162669 0.58582004 0.17230001]
 [0.69804799 0.338117   0.59988499 0.196326  ]
 [0.76785726 0.34902603 0.51190484 0.16287881]
 [0.69198788 0.34599394 0.58626751 0.24027357]
 [0.73446047 0.37367287 0.5411814  0.16750853]
 [0.70953708 0.28008043 0.61617694 0.1960563 ]
 [0.70631892 0.37838513 0.5675777  0.18919257]
 [0.80377277 0.55160877 0.22064351 0.0315205 ]
```

Figure 1.2.4. Normalised features

4. **Design the deep neural network (DNN) architecture**
   a. Import the methods models and layers from the library `tensorflow.keras`
   b. Initialize the neural network model using the method `models.Sequential()`
   c. Calculate the `feature_dim` paprameter as `X_train_transformed.shape[1]`
   d. Using the method `modelName.add()`, add the following layers, using the method `layers.LayerName`
      - 1st Hidden layer: Dense layer with 16 nodes, activation function = "ReLu" and input dimension = feature_dim
      - A Dropout layer, with dropout rate = 0.2
      - 2nd Hidden layer with 16 nodes and activation function = "ReLu"
      - Another Dropout layer, with dropout rate = 0.2
      - Output layer, of type Dense, with units = 3 and activation = "Softmax"
   e. Compile the network using the method `networkName.compile` with the following arguments:
      - `loss = "categorical_crossentropy"`
      - `optimizer = "adam"`
      - `metrics = ["accuracy"]`
   f. Display the network architecture using the method `networkName.summary()`

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 16)	80
dropout_3 (Dropout)	(None, 16)	0
dense_7 (Dense)	(None, 16)	272
dropout_4 (Dropout)	(None, 16)	0
dense_8 (Dense)	(None, 3)	51

Total params: 403 (1.57 KB)
Trainable params: 403 (1.57 KB)
Non-trainable params: 0 (0.00 B)

Figure 1.2.5. DNN network

5. **Training**

Train the model using the method `model.fit()` applied on the normalised features (X_train_transformed) and the encoded labels (y_train_encoded) with the following training parameters:

- `Number of epochs = 50`
- `Batch size = 32`
- `Validation_split = 0.3`

Let the variable history receive the result of this function.

Visualise the evolution of loss, accuracy on the training and on the validation sets at each epoch

```
Epoch 1/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 153ms/step - accuracy: 0.2679 - loss: 1.0990 - val_accuracy: 0.2000 - val_loss: 1.1169
Epoch 2/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.2828 - loss: 1.0984 - val_accuracy: 0.2000 - val_loss: 1.1131
Epoch 3/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.3426 - loss: 1.0993 - val_accuracy: 0.2000 - val_loss: 1.1096
Epoch 4/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.4636 - loss: 1.0550 - val_accuracy: 0.2000 - val_loss: 1.1065
Epoch 5/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.3732 - loss: 1.0643 - val_accuracy: 0.2000 - val_loss: 1.1038
Epoch 6/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.4220 - loss: 1.0547 - val_accuracy: 0.2000 - val_loss: 1.1009
Epoch 7/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.3498 - loss: 1.0536 - val_accuracy: 0.2000 - val_loss: 1.0966
```

Figure 1.2.6. DNN network's training progress

6. Test the learnt model
   a. Calculate and print the predicted probabilities on the test set by apply the method `networkName.predict()` on the normalised test features X_test_tranform

```
Predicted probabilities:
 [[0.27064735 0.40307415 0.32627845]
 [0.577006   0.2258592  0.19713476]
 [0.21117155 0.43103355 0.3577949 ]
 [0.27113855 0.40237942 0.32648206]
 [0.26262787 0.40852758 0.32884458]
 [0.5595337  0.23700972 0.2034564 ]
 [0.31526574 0.38141683 0.3033174 ]
 [0.24309963 0.41720763 0.33969277]
 [0.23153098 0.42513192 0.34333703]
 [0.28923738 0.39393112 0.31683153]
 [0.25598764 0.40959355 0.3344188 ]
```

Figure 1.2.7. Output probabilities for each class (by row)

b. For each row, the predicted label corresponds to the rank of the highest probability. Convert the predicted probabilities into labels using the method `np.argmax()` with `axis = 1` (i.e. get the maximum value for each row)

```
Predicted labels:
 [1 0 1 1 1 0 1 1 1 1 1 0 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 0 0 0 0 1 0 0 1 1
 0 0 0 1 1 1 0 0]
```

Figure 1.2.8. Predicted labels

c. Calculate and print the confusion matrix and the classification report
Hint! Import the methods `confusion_matrix` and `classification_report` from the library `sklearn.metrics`

```
Confusion matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0 13  0]]
Classification report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.50      1.00      0.67        13
           2       0.00      0.00      0.00        13

    accuracy                           0.71        45
   macro avg       0.50      0.67      0.56        45
weighted avg       0.57      0.71      0.61        45
```

Figure 1.2.9. Confusion matrix and classification report

7. Go back to steps 4 and 5 and try out another configuration of the network using different parameters.

Part II- Convolutional Neural Networks (CNN)

Tutorial II: Feature extraction and classification using CNN

In this tutorial, we use CIFAR images dataset to train a CNN network, using *Tensorflow* and *Keras* prebuilt functions (see the file U*B_AI_Lecture8_CNN_Example.ipynb*).

1. Import Tensforflow and Keras libraries

```
# Import Tensorflow and Keras
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

import matplotlib.pyplot as plt
```

2. Import and preprocess the CIFAR10 dataset

```
# Download and prepare the CIFAR10 dataset
(train_images, train_labels), (test_images, test_labels) =
datasets.cifar10.load_data()

# Normalize pixel values (between 0 and 255) to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 46s 0us/step
```

Figure 2.1.1. Dowloading the CIFAR dataset

3. Check and visualise the data

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```

Figure 2.1.2. Examples of images of the CIFAR dataset

4. Create the CNN network (The feature extraction part) using a cascade of 2D-convolutional layers (`Conv2D`) with a nonlinear activation function of type `Relu` (Rectified Linear Unit) followed by a 2D-maxpooling layers (`MaxPooling2D`), then display the architecture of the created network using the method `modelName.summary()`

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32,
32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928

Total params: 56,320 (220.00 KB)
Trainable params: 56,320 (220.00 KB)
Non-trainable params: 0 (0.00 B)

Figure 2.1.3 Architecture of the Feature extraction part of the CNN network

5. Add the classifier part using fully-connected (Dense) layers as follows:
   - The first layers results from flattening the output matrix of the feature extractor part of the CNN, using the method `layers.Flatten()`
   - The second Layer in the predictor/classifier part is of type `Dense`, with 64 nodes and an activation of type `relu`
   - The output layer is also of type `Dense` with 10 nodes, which corresponds to the number of classes to predict.

```
model.add(layers.Flatten())

model.add(layers.Dense(64, activation='relu'))

model.add(layers.Dense(10))
```

6. Display the architecture of the full CNN network (Feature extractor & Classifier)

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)
Trainable params: 122,570 (478.79 KB)
Non-trainable params: 0 (0.00 B)

Figure 2.1.4 Architecture of the full CNN network (Feature extractor & Classifier)

7. Compile the model using the following parameters: Optimiser/training algorithm = "Adam", loss function = "ategorical cross entropy", evaluation metric (on the training set) = "accuracy"

```
model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategoricalCross
entropy(from_logits=True), metrics=['accuracy'])
```

8. Set the number of epochs to 10 and train the model using{X_test,y_test}as validation set. **Note** that training is performed on {X_train, y_train} whereas the validation set is used to assess the evaluation metric, accuracy in this case during training.

```
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

9. Visualise the accuracy curve during training

```
plt.plot(history.history['accuracy'], label='train_accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

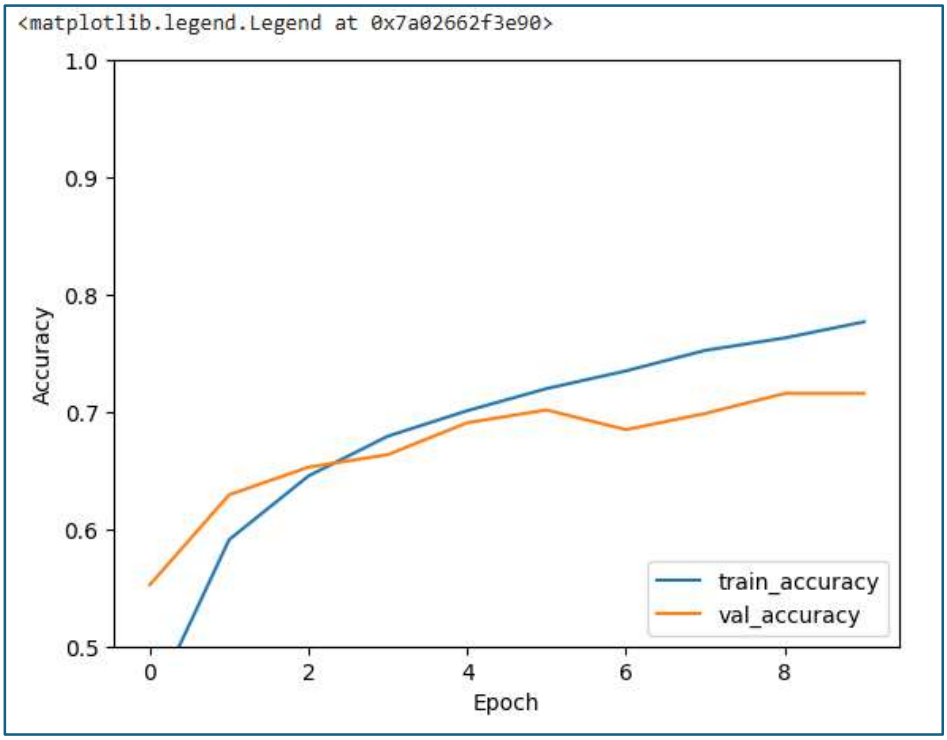
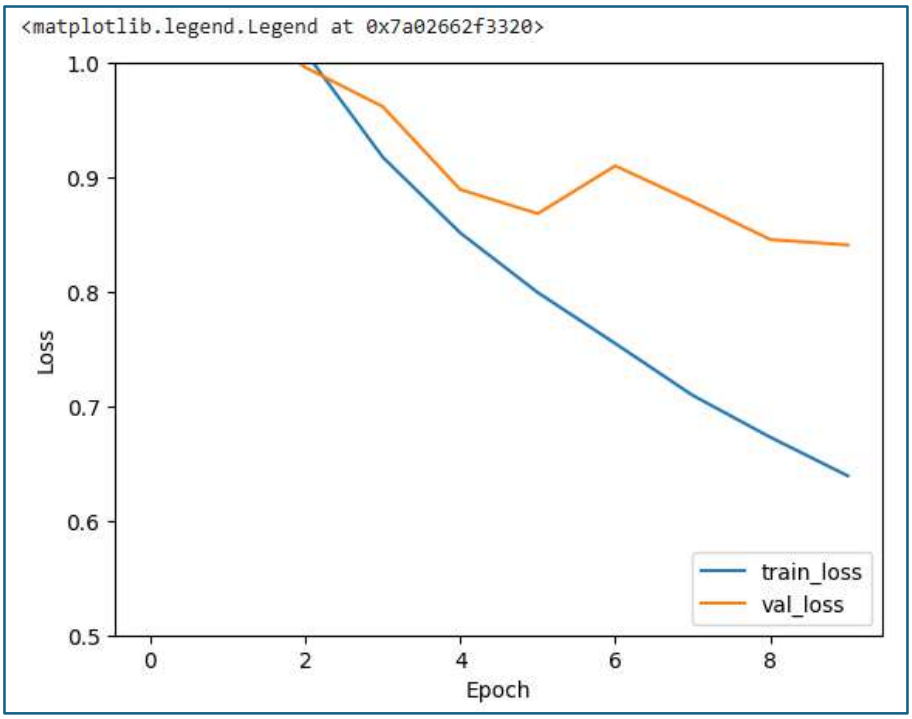


Figure 2.1.5 Evolution of Accuracy on the training and validation set during training epochs

10. Visualise the loss curve during training

```
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label = 'val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```



11. Evaluate the model on the validation set (Here we use {test_images, test_labels} as a validation set, but it can different).

```
val_loss, val_acc = model.evaluate(test_images,  test_labels,
verbose=2)

print('Validation loss = ','{0:.2f}'.format(val_loss*100),'%')

print('val accuracy = ','{0:.2f}'.format(val_acc*100),'%')
```

```
313/313 - 4s - 13ms/step - accuracy: 0.7197 - loss: 0.8513
Validation loss =  85.13 %
val accuracy =  71.97 %
```

Figure 2.1.6 Loss and accuracy on the validation set after the end of training

12. Predict the class labels on the test set

```
y_pred = model.predict(test_images)

y_pred

import numpy as np

pred_labels = np.argmax(y_pred, axis=1)
```

13. Claculate the accuracy and generate the classification report on the test set

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

test_accuracy = accuracy_score(test_labels, pred_labels)
print('Accuracy on the test set: %.2f' % (test_accuracy*100))
print(classification_report(test_labels,pred_labels))
```

```
Accuracy on the test set: 71.97
              precision    recall  f1-score   support

           0       0.70      0.80      0.74      1000
           1       0.84      0.83      0.84      1000
           2       0.66      0.60      0.63      1000
           3       0.53      0.55      0.54      1000
           4       0.66      0.71      0.68      1000
           5       0.63      0.59      0.61      1000
           6       0.77      0.81      0.79      1000
           7       0.78      0.74      0.76      1000
           8       0.85      0.77      0.81      1000
           9       0.81      0.79      0.80      1000

    accuracy                           0.72     10000
   macro avg       0.72      0.72      0.72     10000
weighted avg       0.72      0.72      0.72     10000
```

Figure 2.1.7 Accuracy and classification report based on prediction on the test set in CIFAR10 dataset (each class represents an image label)

Exercise #2: Multi-class classification using CNN

In this exercise, we aim at designing and training a convolutional neural network (CNN) to classify MNIST dataset images that contain handwritten digits (from 0 to 9).

Steps

1. **Setup-** Import the following libraries: `numpy as np`; `keras`; `layers from keras`; `matplotlib.pyplot as plt`
2. **Data preparation:**
   a. Declare the following parameters: `number_class = 10` and `input_shape = (28,28,1).`
   b. Load the MNIST dataset and split it into training and test datasets.
   Hints!
   Use the result of the method `keras.datasets.mnist.load_data()`
   To the datasets (X_train, y_train), (X_test, y_test)
   c. Display the first sample of the dataset: X_train[0]
   Hint! Apply the method `plt.imshow()`

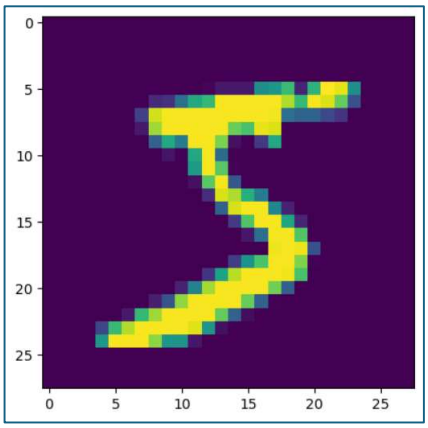


Figure 2.2.1. Example of MNIST dataset for handwritten digits

3. Feature processing

Scale the input features, i.e. the values of the image pixels

a. Divide the values of X_train and X_test by 255 (maximum pixel value)
Hint! You need to convert the pixel values into float to perform real-number division
e.g. `X_train.astype("float32")`

b. Ensure that the image dimension is as declared, i.e. (28,28,1)

Hint! Use the method `np.expand_dim(X_train, -1)`

c. Print the shape and number of the samples in each set (X_train and X_test)

```
x_train shape: (60000, 28, 28, 1)
60000 train samples
10000 test samples
```

Figure 2.2.2. MNIST Dataset composition

4. Build the CNN network

a. Declare a new `modelName` using the method `keras.squential()` with the following layers (use `keras.LayerName` for each layer):

- Input layer: keras.Layer with `shape = input_shape`
- Convolutional layer 1: keras.Conv2D, with Number of filters = 32, kernel size = (3,3), activation = "ReLu"
- Maxpooling layer 1: keras.Maxpooling2D, with pool_size=(2,2)
- Convolutional layer 2: keras.Conv2D, with Number of filters = 64, kernel size = (3,3), activation = "ReLu"
- Maxpooling layer 2: keras.Maxpooling2D, with pool_size=(2,2)
- Fully-connected layer: keras.Flatten
- Dropout layer: keras.Dropout, with dropout rate = 0.5
- Output layer: keras.Dense with number of nodes = number_classes and activation = softmax

b. Display the network's architecture with the method `modelName.summary()`

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)
Trainable params: 34,826 (136.04 KB)
Non-trainable params: 0 (0.00 B)

Figure 2.2.3. CNN network architecture (Feature extractor and classifier (last 3 layers))

5. Training

a. Compile the model using the method `modelName.compile()` with the following parameters: `loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"]`

b. Train the model with the method `modelName.fit()` on X_train and y_train with the following parameters: `batch_size=128, epochs=15, validation_split=0.1`

```
Epoch 1/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.7727 - loss: 0.7379 - val_accuracy: 0.9767 - val_loss: 0.0863
Epoch 2/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 11s 4ms/step - accuracy: 0.9629 - loss: 0.1215 - val_accuracy: 0.9850 - val_loss: 0.0598
Epoch 3/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9725 - loss: 0.0891 - val_accuracy: 0.9893 - val_loss: 0.0455
Epoch 4/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9773 - loss: 0.0709 - val_accuracy: 0.9887 - val_loss: 0.0414
Epoch 5/15
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9810 - loss: 0.0638 - val_accuracy: 0.9887 - val_loss: 0.0410
```

Figure 2.2.4. Evolution of CNN network's training

6. Evaluate the model after learning

Run the method `modelName.Evaluate()` on X_test and y_test, and display the scores obtained, i.e. loss and accuracy:

```
Test loss: 0.024058971554040911
Test accuracy: 0.9915000200271606
```

Figure 2.2.5. Model evaluation scores

7. Calculate and print the confusion matrix and the classification metrics on the test set, using the method `modelName.Predict()` applied on X_test

Hint! The method `modelName.predict()` returns the probability values for each class. To obtain the predicted class, use the method `np.argmax()` on each row of predictions (`axis = 1`)

	precision	recall	f1-score	support
0	0.99	1.00	0.99	980
1	0.99	1.00	0.99	1135
2	0.99	0.99	0.99	1032
3	0.99	1.00	0.99	1010
4	0.99	1.00	1.00	982
5	0.99	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

Figure 2.2.6. Classification report for the CNN model applied on MNIST dataset (each class represents a handwritten digit)

Exercise #3: Feature extraction and class prediction for image using a pretrained CNN model (VGG16)

The goal of this exercise is to leverage pretrained CNN models, such VGG16, to carry out: firstly feature extraction and then class prediction. VGG16 is a **pretrained model**, i.e. that was learnt by training over a huge dataset, e.g. ImageNet. This implies that is ready to be used, without any further training. In this exercise, we aim at discovering how the feature extractor and the classifier parts contained in this model can be re-used, by applying it to a sample image.

1. Setup : Import the necessary libraries, including VGG16

```
import keras
from keras.applications.vgg16 import VGG16
from keras.applications.vgg16 import preprocess_input
import numpy as np
```

2. Feature extractor

a. Declare the Feature Extractor with the method VGG16 using the following parameters:

- `weights= "ImageNet"` → This means that we will use the use the weights
- `include_top = "False"` → Use only the feature extraction part

b. Display the feature extractor network, using the method
`featureExtractorName.summary()`

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_14 (InputLayer)	(None, None, None, 3)	0
block1_conv1 (Conv2D)	(None, None, None, 64)	1,792
block1_conv2 (Conv2D)	(None, None, None, 64)	36,928
block1_pool (MaxPooling2D)	(None, None, None, 64)	0
block2_conv1 (Conv2D)	(None, None, None, 128)	73,856
block2_conv2 (Conv2D)	(None, None, None, 128)	147,584
block2_pool (MaxPooling2D)	(None, None, None, 128)	0
block3_conv1 (Conv2D)	(None, None, None, 256)	295,168
block3_conv2 (Conv2D)	(None, None, None, 256)	590,080
block3_conv3 (Conv2D)	(None, None, None, 256)	590,080
block3_pool (MaxPooling2D)	(None, None, None, 256)	0
block4_conv1 (Conv2D)	(None, None, None, 512)	1,180,160
block4_conv2 (Conv2D)	(None, None, None, 512)	2,359,808
block4_conv3 (Conv2D)	(None, None, None, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, None, None, 512)	0
block5_conv1 (Conv2D)	(None, None, None, 512)	2,359,808
block5_conv2 (Conv2D)	(None, None, None, 512)	2,359,808
block5_conv3 (Conv2D)	(None, None, None, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, None, None, 512)	0

Total params: 14,714,688 (56.13 MB)
Trainable params: 14,714,688 (56.13 MB)
Non-trainable params: 0 (0.00 B)

Figure 2.3.1. VGG16-Feature extractor network

3. Application of the feature extractor

a. Load the image provided in the lab material *"elephant.jpg"* to Google Colab or Jupyter notebooks

b. If you use Google Drive, use the following code to upload the image to Keras and visualise it:

```
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive/')
img_path = '/content/drive/My Drive/elephant.jpg'
img = keras.utils.load_img(img_path, target_size=(224, 224))
plt.imshow(img)
plt.title("Input image")
plt.show()
```

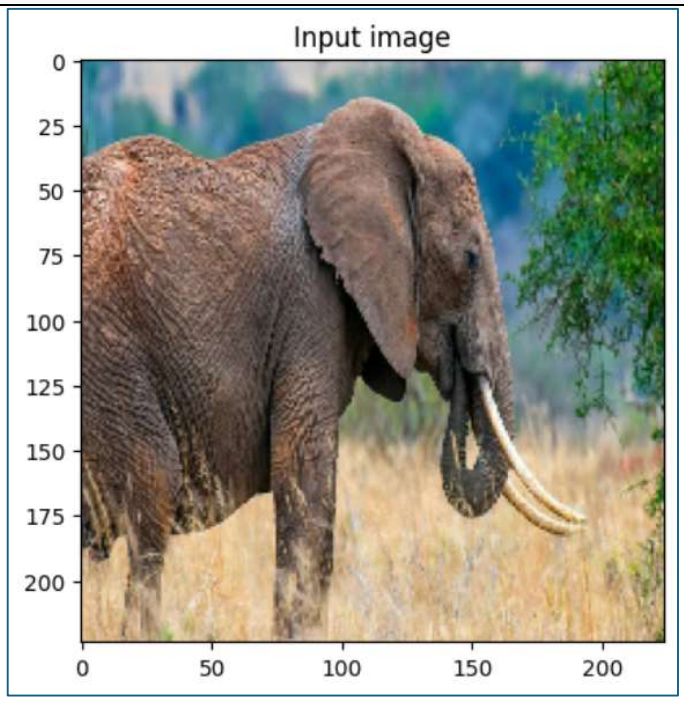


Figure 2.3.2. Input image

c. Prepare the image by reshaping and preprocessing:

- Convert the image to an array using the method `keras.utils.image_to_array()`
- Reshape the image by adding a new dimension, using the method `np.expand_dims()` with `axis = 0`
- Preprocess the image, using VGG16 built-in method `preprocess_input()`

Hint! Check the setup libraries for VGG16

d. Extract the feature from the preprocessed image (x) using the method `featureExtractorName.predict()` and print the shape of the extractec features

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 559ms/step
(1, 7, 7, 512)
```

Figure 2.3.3. Shape of the extracted features

e. Reshape the extracted feature to a 2-D matrix (features.shape[0]* features.shape[1]* features.shape[2], features.shape[3]) and plot the result as an image

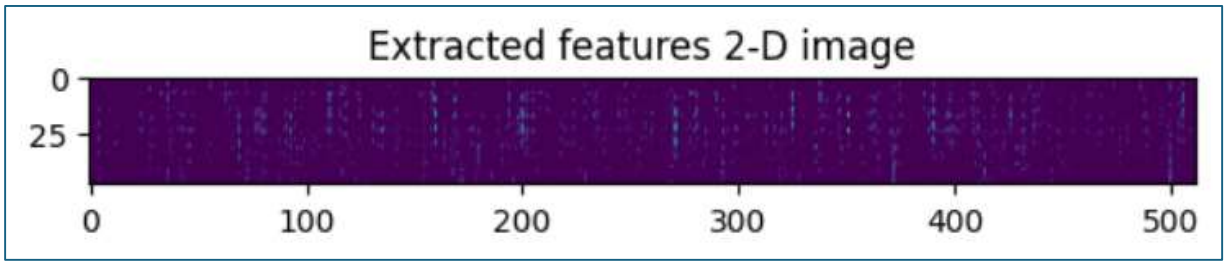


Figure 2.3.4. Representation of the extracted features as a 2-D image

4. Visualisation of the entire network

a. Declare a `pretrainedModelName` as the result of the method VGG16, with the following parameters "`weights = imageNet`" and "`include_top=True`"

b. Display the entire VGG16 network (feature extractor + classifier) using the method `pretrainedModelName.summary()`

odel: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_7 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102,764,544
fc2 (Dense)	(None, 4096)	16,781,312
predictions (Dense)	(None, 1000)	4,097,000

Total params: 138,357,544 (527.79 MB)
Trainable params: 138,357,544 (527.79 MB)
Non-trainable params: 0 (0.00 B)

Figure 2.3.5. VGG16 entire network (feature extractor & classifier (last 4 layers))

5. Use the VGG16 model to classify images

a. Apply the method `pretrainedModelName.predict()` on the extracted features x

b. Print the result, maximum value of probability and the corresponding class

Hint! To decode probability values into classes, use the following method:

```
keras.applications.vgg16.decode_predictions(y_pred,
top=numberOfTopClasses)[0]
```

where y_pred is the result of predicted probabilities. You can visualise more likely classes by setting `numberOfTopClasses` = 2, 3, 4, etc.

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Predicted probabilities: [('n02504458', 'African_elephant', 0.6828056), ('n01871265', 'tusker', 0.28807828), ('n02504013', 'Indian_elephant', 0.029112997)]
Predicted class: [('n02504458', 'African_elephant', 0.6828056)]
```

Figure 2.3.6. Top 3 predicted probabilities and most likely class